

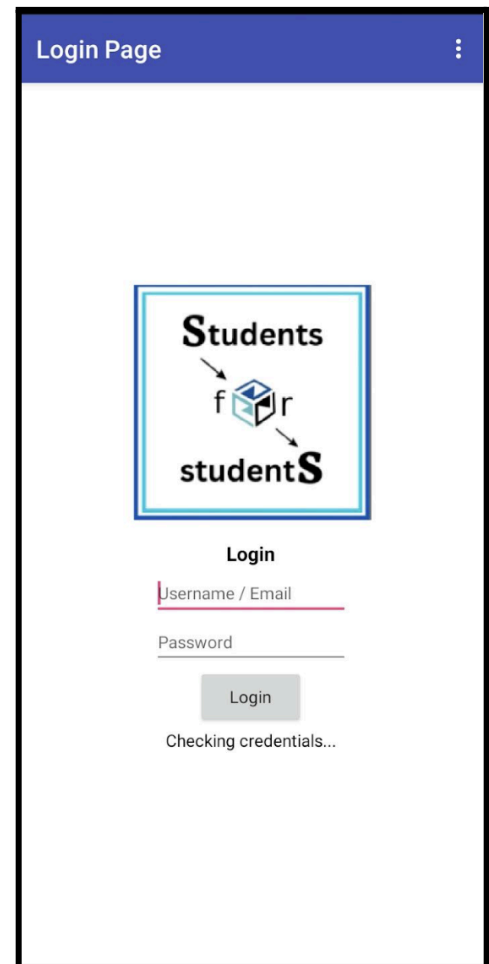
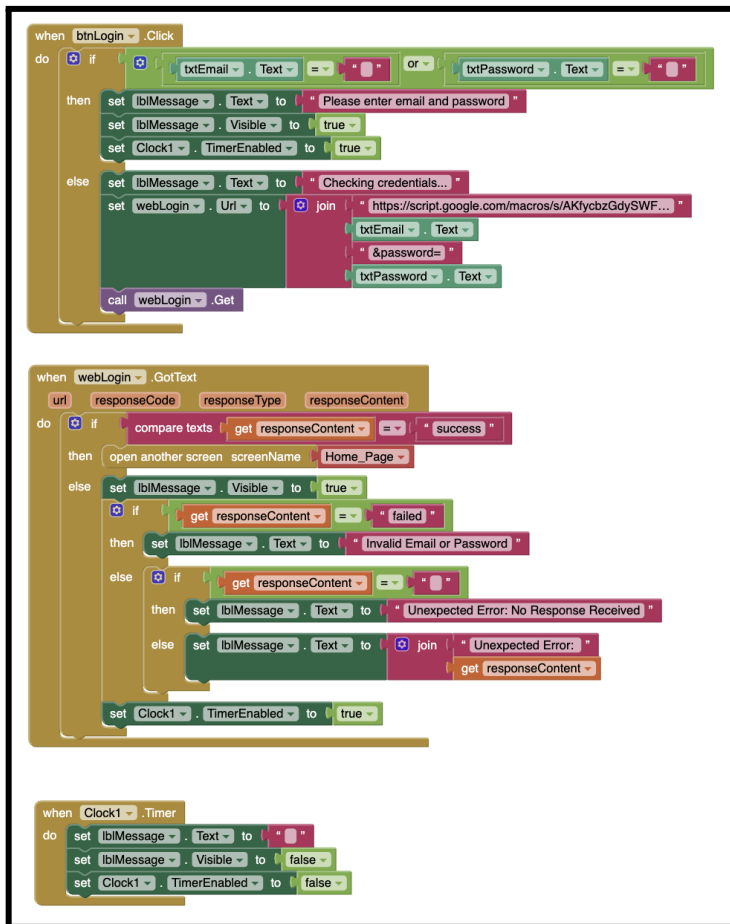
Criterion C: Development

Complexity List:

1. Login Verification system
2. Search Students by Unique ID and display information
3. Compatibility algorithm that recommends the best tutor based on the student's preferences
4. Matchup ID created and updated in sheets data
5. Lists all matched-up tutors

Code

Login Verification



The login system ensures that only users who enter valid credentials can access the application. The input is checked against credentials stored in Google Sheets which are managed by the client. When a user enters their email, and password and clicks the Login button, the API request URL is dynamically generated by appending user inputs,

sending them as query parameters to the authentication script. This enables credential verification by comparing the input against Google Sheets data stored by the client. This is facilitated through the creation of a custom-written Google Apps Script that allows for reading/updating data (See Appendix B). This URL is assigned to webLogin.Url, and the webLogin.Get method is called to send the request. The response is returned as a string, which can either be “success”, “fail” or “”. Then I used an if statement to redirect the user to the home page or display an error message.

I **chose** to use Apps Script to read from Google Sheets **instead** of local authentication since it allows centralized credential management. This way the credentials are accessible and editable to the client. Additionally, string comparison **instead** of hashed authentication was chosen for ease of implementation, as encryption was not a client requirement.

Search Student by ID

```

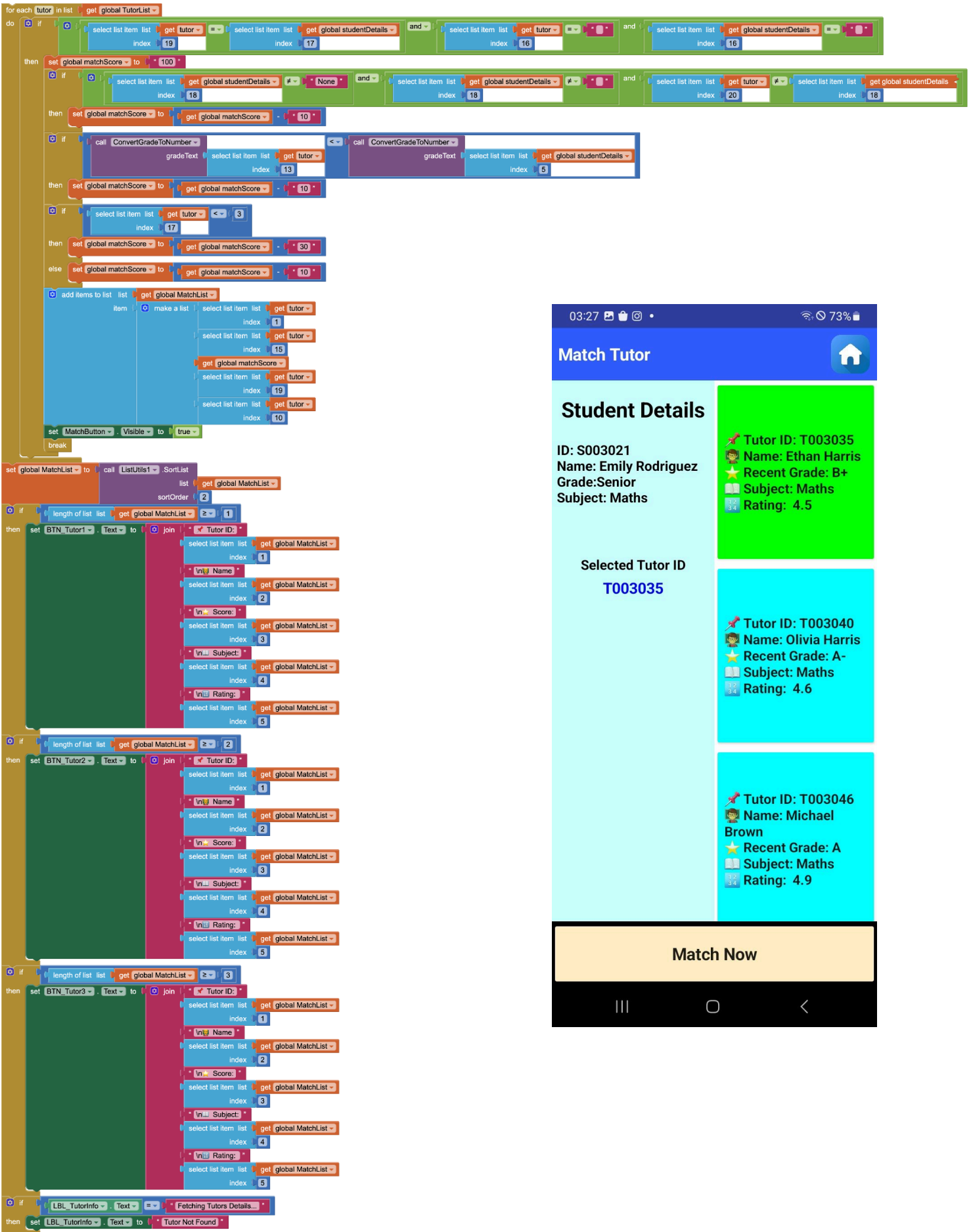
when Btn_ViewStudent .Click
do
  set Lbl_StudentDetails .Text to "Searching..."
  set BTN_CheckMatch .Visible to false
  if is a list? thing get global studentList
  then
    for each tempList in list get global studentList
    do
      if TB_SearchStudent .Text = select list item list get tempList index 15 and select list item list get tempList index 16 = 
      then
        set Lbl_StudentDetails .Text to join (
          " Name: "
          select list item list get tempList index 3
          " \nGrade: "
          select list item list get tempList index 5
          " \nSubject: "
          select list item list get tempList index 17
          " \nEmail: "
          select list item list get tempList index 4
          " \nGender: "
          select list item list get tempList index 19
        )
        set BTN_CheckMatch .Visible to true
        break
      if Lbl_StudentDetails .Text = "Searching..."
      then
        set Lbl_StudentDetails .Text to "Student Not Found"
      else
        set Lbl_StudentDetails .Text to "Error Data Not Loaded"
    end
  end
end
  
```

The search functionality allows the user to find a student or tutor by entering a unique Student ID. When a Student ID is inputted, the system retrieves the dataset from Google Sheets and converts it into a list. Then it iterates through the list using a for each loop(a convenient way of iteration through a read-only loop), extracting each row and checking the value in the ID column against the user's input using a conditional statement.

If the input ID matches the value, the system extracts the relevant columns (e.g., name, subject, and availability) and displays them in a structured format. The select list item block is used to extract the ID column for verification and additional fields for display.

I **chose** a for each loop **instead** of an index-based loop due to its readability and ease of implementation, allowing the system to dynamically scan all records without requiring explicit index calculations. This approach is effective given the relatively small dataset size. I also used a list-based filtering approach to accommodate the future addition of columns and efficiency. Using string-based ID comparison **instead** of numerical indexing allows for the use of the alphanumerical Tutor/Student IDs used by the client.

Compatibility Algorithm

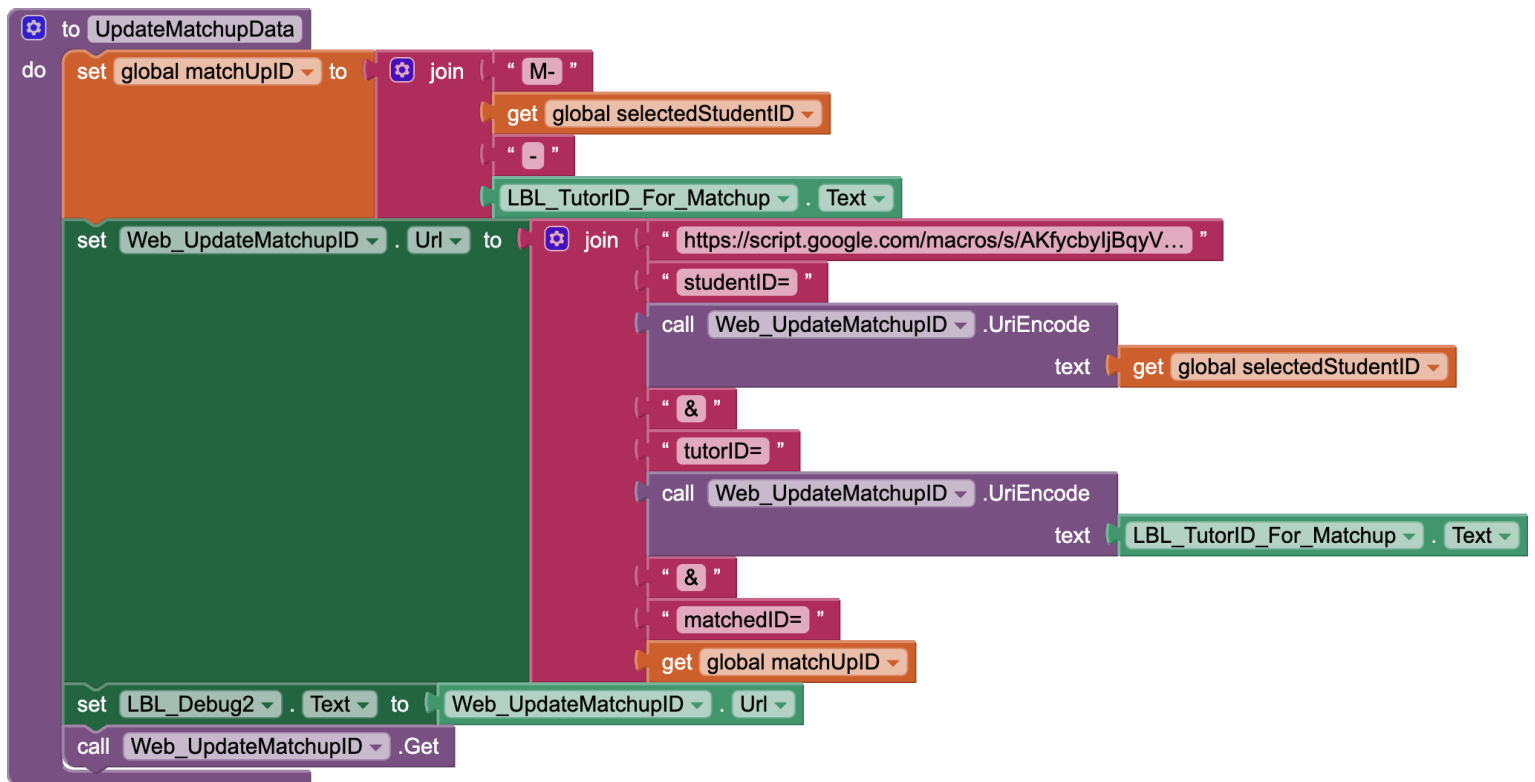


The compatibility algorithm automates the student-tutor matching process by evaluating multiple factors including subject, gender, tutor rating, and grade level. The App first sets the compatibility score to 100. Then it retrieves tutor and student data from Google Sheets and applies a matching algorithm. The String “Grade Level” is converted to an integer to perform comparison operations.

A nested for each loop iterates through both the student’s preference and unmatched tutor lists excluding all tutors with a different Subject. Within the loop, the “select list item list” block extracts necessary attributes, which are compared against predefined criteria using conditional statements. The scoring system assigns penalties based on mismatched criteria (See Criterion B), ensuring higher compatibility scores for better tutor-student matches. Finally, the tutors with the three highest scores are displayed and can be matched up with the students.

I chose to use a weighted scoring system **instead** of first-come-first-serve matching to ensure the most suitable tutor is assigned. Nested loops were used **instead** of brute-force searching, improving structured iteration and efficiency. Conditional statements were used to make the algorithm adaptable to future enhancements, such as additional matching criteria.

Create and Update Matchup ID



Sample of Tutor and Student Spreadsheet:

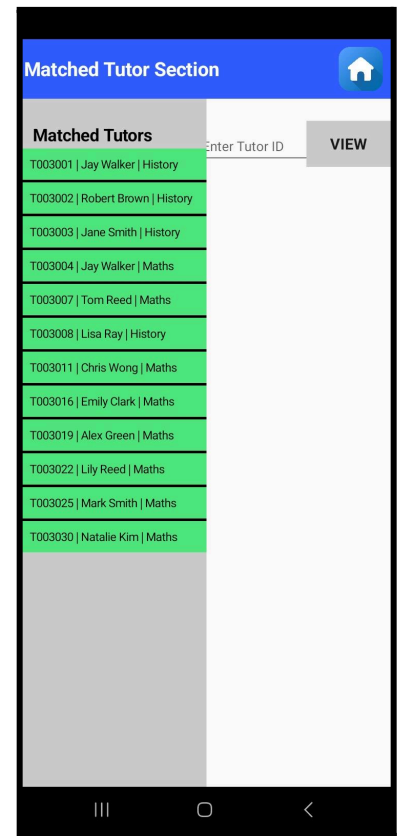
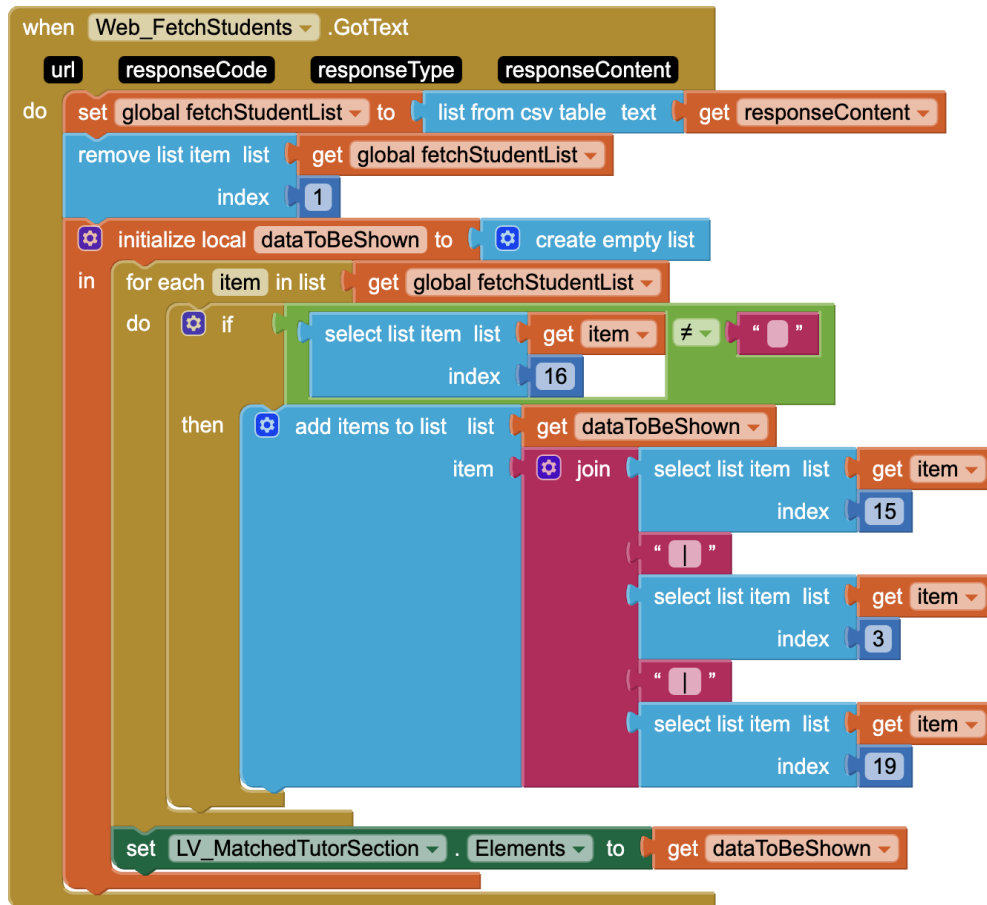
Grade are	Who is your cou	TutorID	MatchedID	Tutor Rating	Interview Done c	Subject	Gender	
	Mr. Loy Jayne	T003001	M-S003001-T00	5.0	Yes	History	Male	
an	Ms John Doe	T003002	M-12345-7586	4.2	Yes	History	Female	
	Mr Harry Paul	T003003	M-S003004-T00	4.8	Yes	History	Male	
	Mr. Loy Jayne	T003004	M-S003003-T00	5	Yes	Maths	Male	
	Ms John Doe	T003005		4.2	Yes	History	Male	

is your co	StudentID	MatchedID	Subject	Any Gender Pre	Your Gender
oy Jayne	S003001	M-S003001-T0030	History	Male	Male
ohn Doe	S003002		English	Female	Male
arry Paul	S003003	M-S003003-T0030	History	Male	Male
arah Lane	S003004	M-S003004-T0030	History	Female	Male
ichard Gill	S003005	M-S003005-T0030	Maths	Female	Male

Each student-tutor pair is assigned a unique Matchup ID in the format "M-StudentID-TutorID" to facilitate tracking. When a match is made, the system appends student and tutor IDs using the join block, creating a structured identifier. This ID is stored in the Google Sheets MatchedID column for the Student as well as the Tutor through an Apps Script request.

I chose to use structured Matchup IDs **instead** of randomly generated UUIDs because they simplify retrieval and the client can directly see the TutorID and StudentID when looking at the MatchUp ID. The join block ensures efficient string manipulation. Using Apps Script to query/update the google sheet **instead** of direct local modifications improves system scalability and enables remote access for the client.

Display a List of Matched Tutors



This feature identifies tutors who are currently assigned to students and presents them in a ListView. The system retrieves tutor data and filters out those who don't have a Matchup ID.

A for each loop iterates through the dataset, and a conditional statement checks if a tutor has an assigned Matchup ID. If they do, their details are added to a separate list called data to be shown using the "add items to list" block. The final list is then assigned to the list View: LV_MatchedTutorSection.Elements to update the user interface.

I chose to use list filtering so that the client is able to segregate based on whether they have been matched or not **instead** of the joint spreadsheet for all tutors. Conditional statements dynamically filter out unmatched tutors, ensuring that only actively paired tutors appear in the matched list. This method is preferred **instead** of manual exclusions or fixed filter values since it allows for real-time updates.

Justification for the Range of Tools Used:

The selected techniques optimize three key factors: efficiency (minimizing processing time), scalability (handling increasing student numbers), and maintainability (ensuring the client can manage data easily). List-based data structures and iterative loops were used throughout to process data, while conditional filtering ensures that only required information is displayed. The conditions can be adjusted easily allowing greater flexibility. The unique Matchup identifier prevents conflicting matchup information. Apps Scripts integration with Google Sheets eliminates the need for manual data handling, reducing human error previously faced by the client. Google Sheets was chosen over Firebase because the client prefers direct access to the data. A traditional database would offer faster queries, but Sheets provides better accessibility for non-technical users.

I chose to automate tutor matching instead of using manual selection to optimize decision-making based on select criteria that can be adjusted. Fetching and displaying updated tutor lists ensures users receive real-time feedback on availability, improving user experience. Using URI encoding instead of direct text input increases possible inputs, making the system adaptable for future upgrades.

Word count: 981

Works referenced:

Ward, Chris. "ProfessorCad: Tips & Tricks." *Professorcad.co.uk*, 2015,

www.professorcad.co.uk/appinventortips#Loops. Accessed 10 Mar. 2025.

"Custom Functions in Google Sheets." *Google for Developers*, 2025,

developers.google.com/apps-script/guides/sheets/functions. Accessed 10 Mar. 2025.